Parul® University

NAAC A++

Chapter-6: AWS Security, Monitoring, Scaling and Billing

Jayshree Ranoliya
Assistant Professor
Artificial Intelligence and Data Science Department

Parul® University

NAAC A++

Content

INDEX

Introduction to Cloud Security & Management

- Cloud computing requires robust security, effective monitoring, and flexible scaling.
- AWS provides tools to manage user access, monitor performance, and optimize cost.
- These elements ensure reliability, efficiency, and compliance in cloud environments.

Key Idea: Secure, monitor, and scale your cloud resources intelligently.

AWS Shared Responsibility Model

- **AWS's Responsibility:** Security *of* the cloud (hardware, software, network, facilities).
- **Customer's Responsibility:** Security *in* the cloud (data, access management, configurations).
- Together, they ensure total protection.

Example: AWS secures EC2 infrastructure, but you manage OS patches and IAM permissions.

AWS Identity and Access Management (IAM) Overview

- IAM controls **who can access** AWS resources and **what actions** they can perform.
- Provides **fine-grained access control** across AWS services.
- Core elements: Users, Groups, Roles, and Policies.

Benefit: Ensures only authorized entities interact with critical resources.

.

IAM Users

- Represent individuals or applications accessing AWS.
- Each user has unique credentials (username, password, access keys).
- Permissions are granted directly or via groups.
- **Best Practice:** Grant only the permissions necessary (least privilege).

IAM Groups

- Logical grouping of users with shared permissions.
- Simplifies permission management—assign policies once to the group.
- Examples:
  - “Developers” → Access to EC2 and S3
  - “Admins” → Full access
  - “Auditors” → Read-only access

Result: Easier scalability and consistent access control.

IAM Roles

Roles provide **temporary security credentials** for AWS services or users.

Commonly used by EC2 instances, Lambda functions, or cross-account access.

No permanent credentials — improves security posture.

Example: EC2 role granting access to S3 bucket for backups.

IAM Policies

Define *what actions* are allowed or denied on *which resources*.

Written in **JSON** format.

Can be attached to users, groups, or roles.

Example Policy:

```
{
 "Effect": "Allow",
 "Action": ["s3:GetObject"],
 "Resource": ["arn:aws:s3:::mybucket/*"]
}
```

Implementing IAM Policies

Use **AWS Management Console**, **CLI**, or **CloudFormation**.

Steps:

1. Define the scope and actions (e.g., “Allow EC2 StartInstances”).
2. Apply least privilege.
3. Assign to the correct identity (user, group, or role).

Continuously **audit policies** and **remove unused permissions**.

IAM Best Practices

Enable **Multi-Factor Authentication (MFA)** for all accounts.

Avoid using root credentials for daily operations.

Rotate passwords and access keys regularly.

Use **AWS CloudTrail** to monitor login and API activity.

Regularly review IAM reports and credential usage.

AWS CloudWatch Overview

- AWS CloudWatch monitors applications and system performance.
- Collects **metrics, logs, and events** from AWS resources.
- Supports custom dashboards and alarms for automation.
- Enables proactive issue detection before users are impacted.

CloudWatch Metrics and Alarms

- Metrics include CPU, memory, disk I/O, and network activity.
- Alarms notify when metrics exceed thresholds.
- Actions: send SNS notifications, execute Lambda, or trigger Auto Scaling.
- Example: Trigger alarm if EC2 CPU > 80% for 5 minutes.

CloudWatch Logs and Dashboards

- Collects logs from EC2, Lambda, and other AWS services.
- Centralized log storage and real-time analysis.
- Dashboards visualize metrics and logs together.
- Helps identify performance bottlenecks and security issues.

Elastic Load Balancing (ELB)

- Distributes incoming traffic across multiple targets (EC2, containers, IPs).
- Improves application **availability and fault tolerance**.
- Types of ELB:
  a. Application Load Balancer (HTTP/HTTPS)

  b. Network Load Balancer (TCP/UDP)

  c. Gateway Load Balancer (third-party appliances)

Benefits of Elastic Load Balancing

- Provides **high availability** and **automatic failover**.
- Performs **health checks** to route traffic only to healthy instances.
- Scales automatically with traffic volume.
- Works seamlessly with **Auto Scaling Groups** for elasticity.

Auto Scaling Overview

- Adjusts compute capacity automatically to match demand.
- Ensures performance while optimizing cost.
- Reduces manual intervention in resource management.
- Common for EC2, ECS, and DynamoDB services.

Auto Scaling Components

- **Launch Configuration / Launch Template:** Defines instance details.
- **Auto Scaling Group (ASG):** Logical group of EC2 instances managed together.
- **Scaling Policies:**

- *Target Tracking* (maintain average CPU)

- *Step Scaling* (react to thresholds)

- *Scheduled Scaling* (predefined time events)

Scaling Strategies

- **Vertical Scaling:** Add more resources (CPU/RAM) to an existing instance.
- **Horizontal Scaling:** Add more instances to distribute load.
- **Elastic Scaling:** Combines both approaches dynamically.
- ELB + Auto Scaling = Highly available, fault-tolerant architecture.

Monitoring Scaled Architectures

- Use **CloudWatch metrics** to monitor scaling performance.
- Set alarms for key indicators like CPU, latency, and request count.
- **CloudTrail** logs API actions for auditing scaling events.
- Helps detect anomalies and optimize scaling policies.

Cloud Economics and Billing Overview

- AWS billing model is **pay-as-you-go**.
- No upfront investment or hardware costs.
- You pay only for what you use — compute, storage, and network.
- Encourages efficiency and experimentation.

AWS Pricing Models

- **On-Demand:** Pay for compute by the second/hour (no commitments).
- **Reserved Instances:** 1- or 3-year commitment for significant discounts.
- **Spot Instances:** Lowest cost; use spare capacity (variable availability).
- **Savings Plans:** Flexible alternative to reserved pricing.

AWS Cost Management Tools

- **AWS Cost Explorer:** Analyze past spending and usage trends.
- **AWS Budgets:** Set cost thresholds and receive alerts.
- **AWS Pricing Calculator:** Estimate cost before deployment.
- **Cost and Usage Reports (CUR):** Detailed cost tracking per service.

Optimizing AWS Costs

- Use **Auto Scaling** to eliminate idle resources.
- Choose appropriate instance types and regions.
- Implement **Storage Lifecycle Policies** for S3.
- Monitor cost drivers regularly using **Cost Explorer**.
- Leverage **Reserved and Spot Instances** where applicable.

Integrating Security, Scaling, and Monitoring

- IAM ensures secure access to resources.
- CloudWatch monitors infrastructure and triggers Auto Scaling.
- ELB balances workload for fault tolerance.
- Cost management tools control spending.
- Together, these create a **secure, scalable, and cost-efficient ecosystem.**

Summary and Key Takeaways

- **IAM:** Manage identities and permissions securely.
- **CloudWatch:** Monitor metrics, logs, and set automated responses.
- **ELB + Auto Scaling:** Ensure performance and availability.
- **Billing Tools:** Optimize cost and prevent overspending.
- A well-architected AWS solution balances **security, scalability, and efficiency.**

References

1. Amazon Web Services, *AWS Identity and Access Management (IAM) User Guide*, 2024.

2. Amazon Web Services, *Amazon CloudWatch Documentation*, 2024.

3. Amazon Web Services, *Elastic Load Balancing User Guide*, 2024.

4. Amazon Web Services, *AWS Auto Scaling Documentation*, 2024.

5. Amazon Web Services, *AWS Pricing and Billing User Guide*, 2024.

6. Amazon Web Services, *Well-Architected Framework Whitepaper*, 2024.

Parul® University | NAAC GRADE A++

https://paruluniversity.ac.in/